

Compiladores

Blocos Básicos e Grafos de Fluxo de Controle

Baseado nos slides cedidos pelo Prof. Sandro Rigo (IC-Unicamp)

Prof. Lucas Ismaily

Universidade Federal do Ceará
Campus Quixadá

Introdução

- Representação gráfica do código de 3 endereços é útil para entender os algoritmos de otimização
 - Nós: computação
 - Arestas: fluxo de controle
 - Muito usado em coletas de informações sobre o programa
-

Blocos Básicos

- Seqüência de instruções consecutivas
- Fluxo de Controle:
 - Entra no início
 - Sai pelo final
 - Não existem saltos para dentro ou do meio para fora da seqüência

$t1 = a * a$

$t2 = a * b$

$t3 = b * 3$

$t4 = t2 - t3$

Algoritmo para Quebrar em BBs

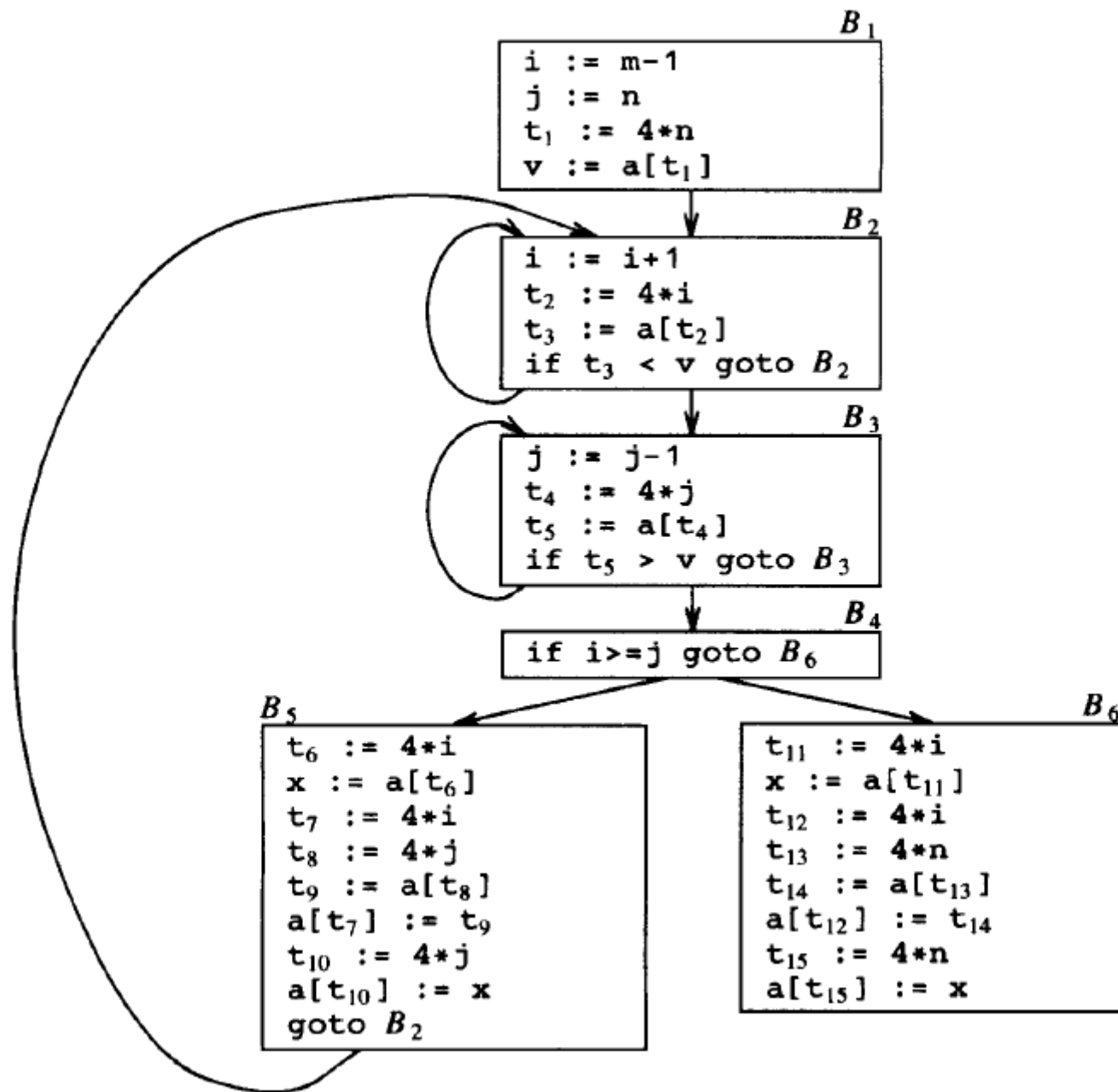
- Entrada: seqüência de código 3 endereços
 - Defina os líderes (iniciam os BBs):
 - Primeira Sentença é um líder
 - Todo alvo de um goto, condicional ou incondicional, é um líder
 - Toda sentença que sucede imediatamente um goto, condicional ou incondicional, é um líder
 - Os BBs são compostos pelos líderes e todas as instruções subsequentes até o próximo líder (exclusive)
-

Quick Sort

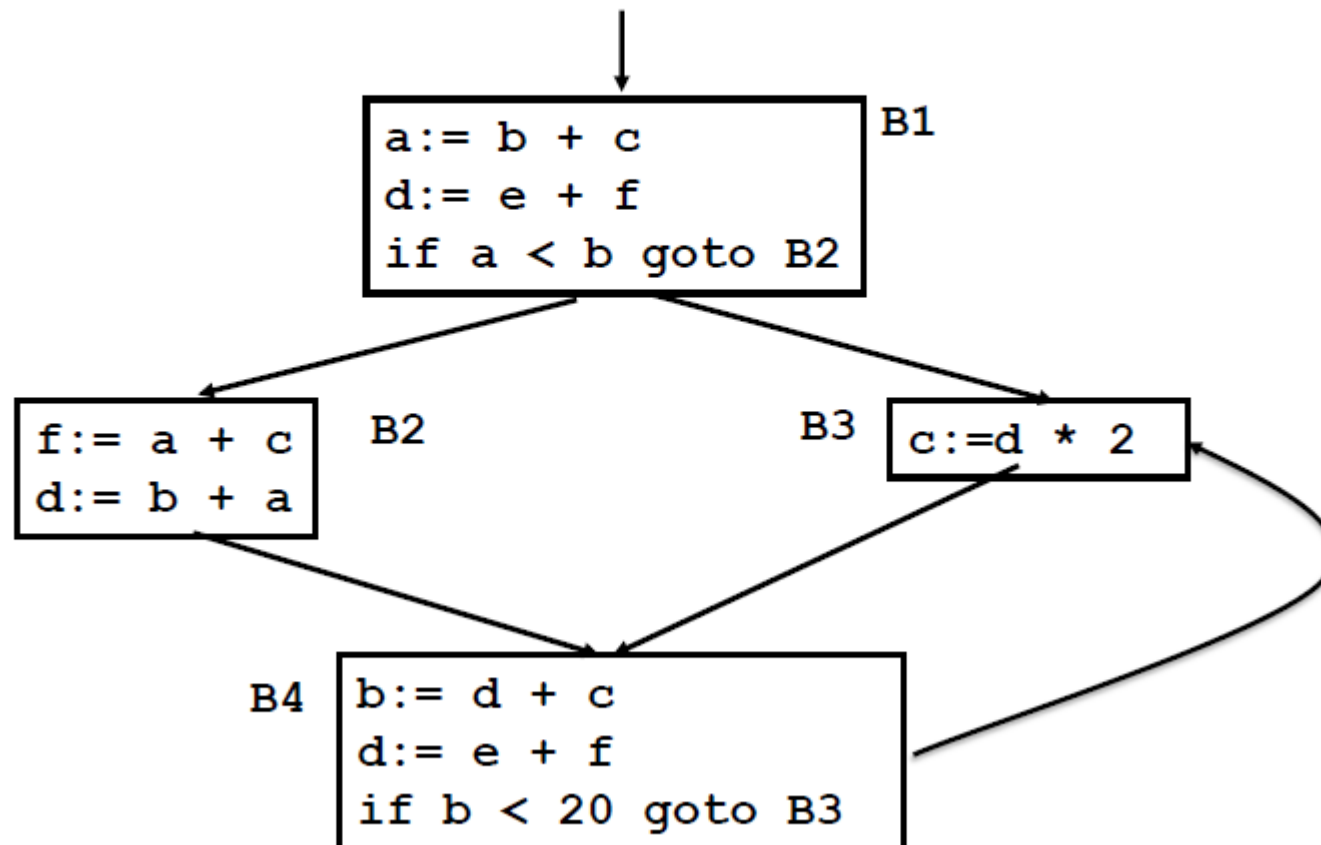
(1) $i := m - 1$	(16) $t_7 := 4 * i$
(2) $j := n$	(17) $t_8 := 4 * j$
(3) $t_1 := 4 * n$	(18) $t_9 := a[t_8]$
(4) $v := a[t_1]$	(19) $a[t_7] := t_9$
(5) $i := i + 1$	(20) $t_{10} := 4 * j$
(6) $t_2 := 4 * i$	(21) $a[t_{10}] := x$
(7) $t_3 := a[t_2]$	(22) $\text{goto } (5)$
(8) $\text{if } t_3 < v \text{ goto } (5)$	(23) $t_{11} := 4 * i$
(9) $j := j - 1$	(24) $x := a[t_{11}]$
(10) $t_4 := 4 * j$	(25) $t_{12} := 4 * i$
(11) $t_5 := a[t_4]$	(26) $t_{13} := 4 * n$
(12) $\text{if } t_5 > v \text{ goto } (9)$	(27) $t_{14} := a[t_{13}]$
(13) $\text{if } i \geq j \text{ goto } (23)$	(28) $a[t_{12}] := t_{14}$
(14) $t_6 := 4 * i$	(29) $t_{15} := 4 * n$
(15) $x := a[t_6]$	(30) $a[t_{15}] := x$

Fig. 10.4. Three-address code for fragment in Fig. 10.2.

Quick Sort



Grafo de Fluxo de Controle (CFG)



Laços

- O que é um laço?
 - Como encontrá-los?
 - Na figura anterior: B3, B4
 - Um laço é um conjunto de nós tais que:
 - Todos os nós são fortemente conectados
 - Têm uma única entrada
 - Um laço que não contém outros laços é um *inner loop*
-

Directed Acyclic Graphs

- Úteis para transformações em BBs
 - Mostra como os valores computados são usados em sentenças subsequentes
 - Common Sub-expression Elimination (CSE)
 - Não confundir com o CFG
 - DAG: representa um BB
 - CFG: Nós são os BBs
-

Directed Acyclic Graphs

- Construção:
 - Folhas são identificadores únicos
 - variáveis, constantes
 - são os valores iniciais das variáveis
 - usa-se índices para não confundir com valor atual

Directed Acyclic Graphs

- Construção:
 - Nós internos são operadores:
 - valores computados
 - O rótulo é o operador associado
 - podem ter uma lista de variáveis associados
 - é o último valor computado para cada uma delas
 - um nó associado a cada sentença
 - filhos representam a última definição dos operandos
-

Exemplo - DAGs

```
(1) t1 := 4 * i  
(2) t2 := a [ t1]  
(3) t3 := 4 * i  
(4) t4 := b [t3]  
(5) t5 := t2 * t4  
(6) t6 := prod + t5  
(7) prod := t6  
(8) t7 := i + 1  
(9) i := t7  
(10) if i <= 20 goto (1)
```

Aplicações

- Detectar sub-expressões comuns
 - automaticamente durante a construção
 - Detectar os identificadores cujos valores são usados no bloco
 - São as folhas
 - Detectar sentenças que geram valores que podem ser usados fora do bloco
 - São aquelas sentenças que geraram $\text{nó}(x) = n$ durante a construção e ainda temos $\text{nó}(x) = n$ ao final
-

Cuidados

- Arrays

- `x := a[i]`
- `a[j] := y`
- `z := a[i]`

- Pode virar

- `x := a[i]`
- `z := x`
- `a[j] := y`

- Ponteiros, problema similar

- `*p=w`
-

-
- Características de um laço
 - Conceitos necessários para otimizações em laços
 - Dominadores, laços redutíveis, laços naturais
-

Laços

- O que é um laço?
 - Um laço é um conjunto de nós tais que:
 - Todos os nós são fortemente conectados
 - De qualquer nó do laço existe um caminho, dentro do laço, para qualquer outro nó do laço
 - Têm uma única entrada
 - Um laço que não contém outros laços é um *inner loop*
-

Dominadores

- Um nó d domina um nó n ($d \text{ dom } n$) se
 - Todo caminho do nó inicial até n passa por d
- Propriedades
 - Reflexiva: todo nó domina ele mesmo
 - Transitiva: $a \text{ dom } b$ e $b \text{ dom } c \Rightarrow a \text{ dom } c$
 - Anti-simétrica: se $a \text{ dom } b$ e $b \text{ dom } a \Rightarrow a = b$
 - A entrada de um laço domina todos os nós do laço

Dominadores

N = conjunto de nós

1 *dom* N

1 *dom*

2 *dom*

3 *dom*

4 *dom*

5 *dom*

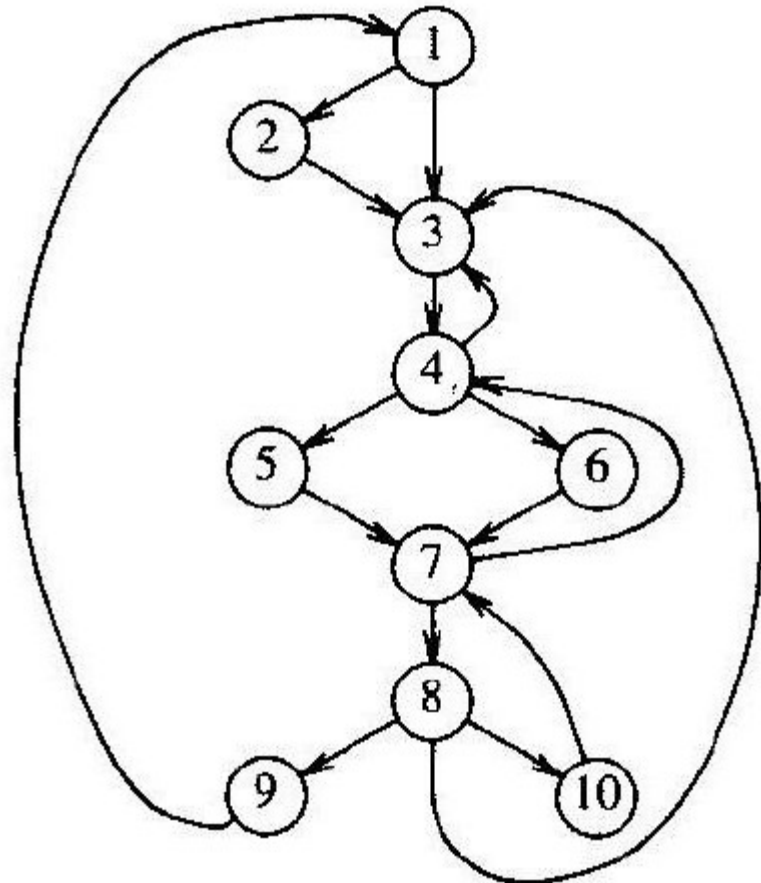
6 *dom*

7 *dom*

8 *dom*

9 *dom*

10 *dom*



Dominador Imediato

- Para $a \neq b$, $a \text{ idom } b$ se e somente se
 - $a \text{ dom } b$
 - Não existe c tal que $c \neq a$ e $c \neq b$ para o qual
 - $a \text{ dom } c$ e $c \text{ dom } b$
 - Dominador imediato é único
 - A informação de dominância imediata pode ser vista como uma árvore
-

Árvore de Dominadores

- O nó inicial é a raiz
 - Cada nó d domina seus descendentes na árvore
 - Relação de dominância: caminhos
 - Cada nó é dominado imediatamente pelo seu pai
 - Idom: arestas
-

Árvore de Dominadores

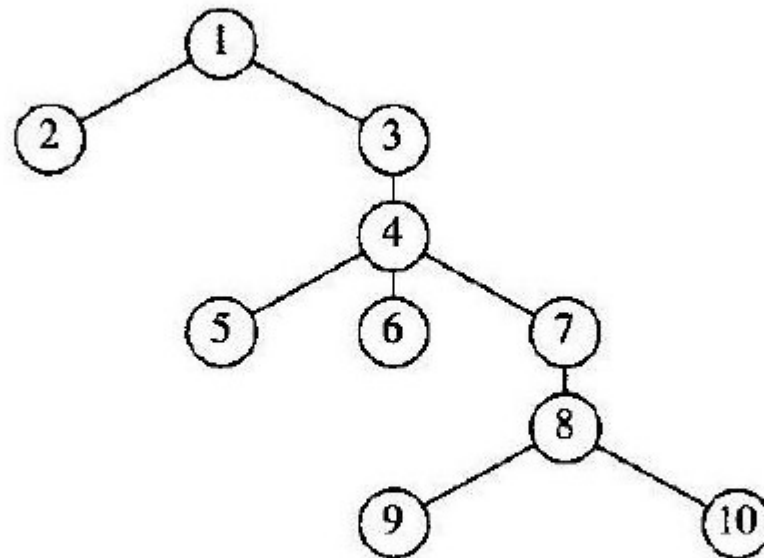


Fig. 10.14. Dominator tree for flow graph of Fig. 10.13.

Algoritmo

- $D(n)$: dominadores de n

```
(1)   $D(n_0) := \{n_0\};$ 
(2)  for  $n$  in  $N - \{n_0\}$  do  $D(n) := N;$ 
      /* end initialization */
(3)  while changes to any  $D(n)$  occur do
(4)    for  $n$  in  $N - \{n_0\}$  do
(5)       $D(n) := \{n\} \cup \bigcap_{\substack{p \text{ a pred-} \\ \text{cessor of } n}} D(p);$ 
```

Laços Naturais

- Propriedades

- Ter um único ponto de entrada
 - Header
- Ter pelo menos um caminho para iterar de volta ao header

- Busca por laços

- Identificar as *back edges*
 - Arestas cuja cabeça domina sua cauda
 - $a \rightarrow b$
 - » a é a cauda
 - » b é a cabeça

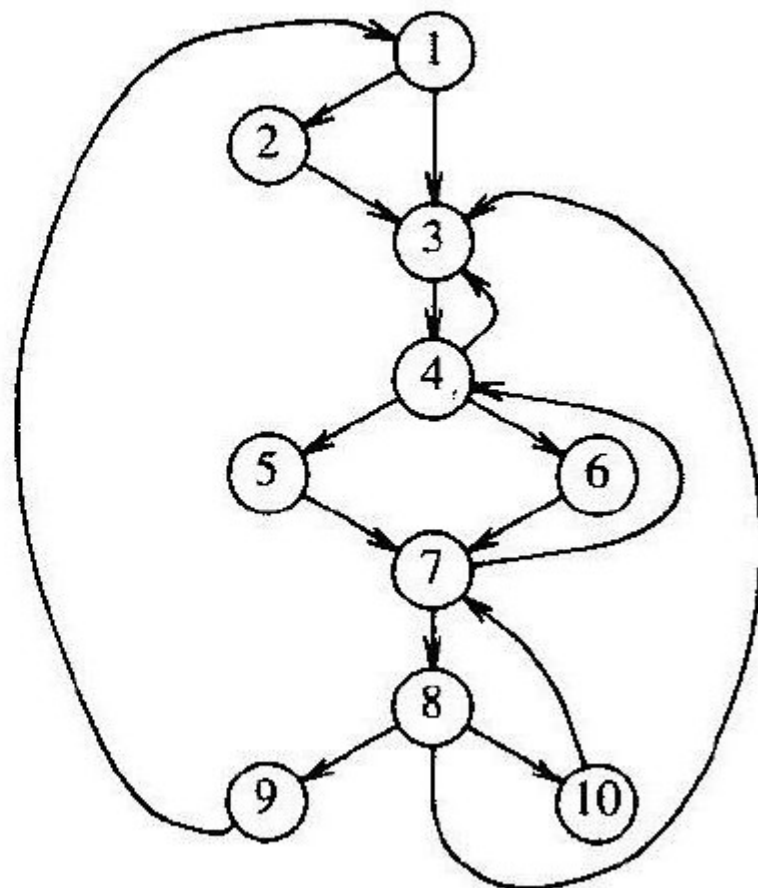
Laços Naturais

- Definição

- Dada uma back edge $n \rightarrow d$
 - Seu laço natural é:
 - d mais o conjunto de nós que podem alcançar n sem passar por d
 - d é o header do laço

Laços Naturais

- Quais os laços naturais das arestas abaixo?
- $10 \rightarrow 7$
- $9 \rightarrow 1$
- $\{4,5,6,7\}$ é um laço?
- $7 \rightarrow 4$



Construindo Laços Naturais

Algorithm 10.1. Constructing the natural loop of a back edge.

Input. A flow graph G and a back edge $n \rightarrow d$.

Output. The set *loop* consisting of all nodes in the natural loop of $n \rightarrow d$.

Method. Beginning with node n , we consider each node $m \neq d$ that we know is in *loop*, to make sure that m 's predecessors are also placed in *loop*. The algorithm is given in Fig. 10.15. Each node in *loop*, except for d , is placed once on *stack*, so its predecessors will be examined. Note that because d is put in the loop initially, we never examine its predecessors, and thus find only those nodes that reach n without going through d . $\square$

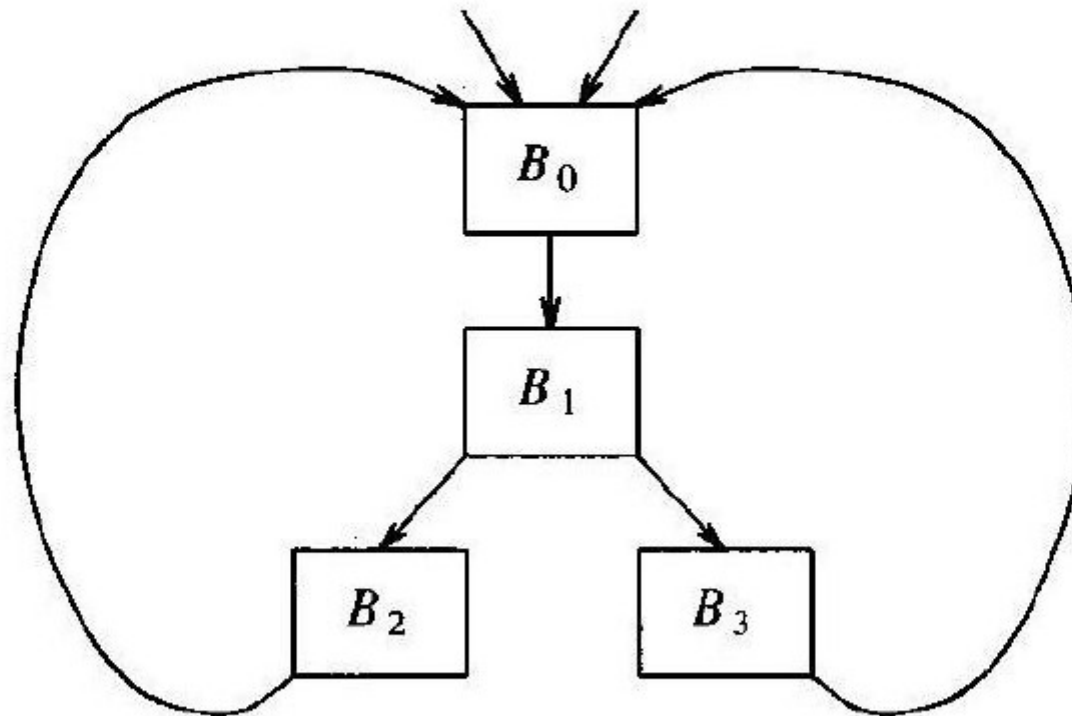
```
procedure insert( $m$ );  
  if  $m$  is not in loop then begin  
     $loop := loop \cup \{m\}$ ;  
    push  $m$  onto stack  
  end;  
  
/* main program follows */  
  
 $stack := \text{empty}$ ;  
 $loop := \{d\}$ ;  
insert( $n$ );  
while stack is not empty do begin  
  pop  $m$ , the first element of stack, off stack;  
  for each predecessor  $p$  of  $m$  do insert( $p$ )  
end
```

Fig. 10.15. Algorithm for constructing the natural loop.

Laços Internos

- Adotando os laços naturais como os únicos laços que trataremos
 - Dois laços que não têm o mesmo header
 - são disjuntos ou
 - um é inteiramente contido no outro
- Se ignorarmos laços com o mesmo header
 - Inner loops são aqueles que não contém outros laços

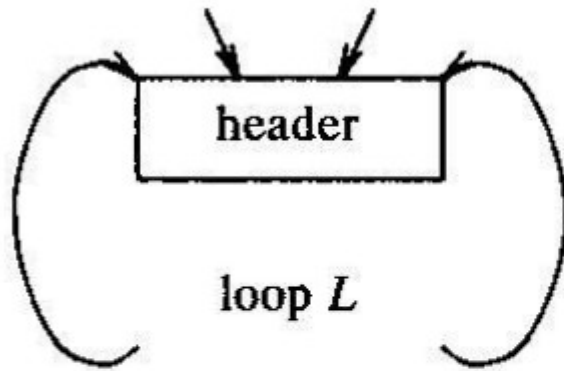
Laços com o mesmo header



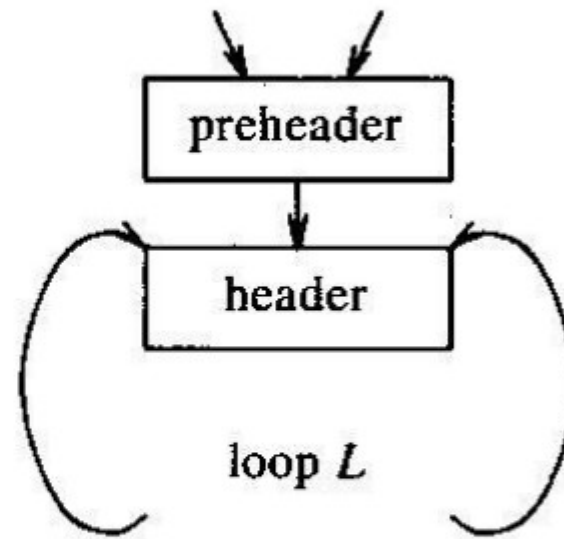
Pre-Headers

- Várias transformações necessitam mover sentenças para fora do laço
 - Antes do header
 - Começamos o tratamento do laço L criando um nó especial
 - Pré-header
 - Tem somente o header como sucessor
 - Todas as arestas externas de L para o header entram agora no pre-header
 - As internas permanecem inalteradas
-

Pre-Headers



(a) Before



(b) After

Fig. 10.17. Introduction of the preheader.

CFG Redutíveis

- Maioria dos CFGs na prática caem nessa classe
 - Dado que temos um programa estruturado
 - Propriedade
 - Não existem saltos para dentro de laços vindo de nós externos ao laço
 - Garante que a entrada de laços é única
 - Definição
 - G é redutível $\Leftrightarrow$ as aretas podem ser particionadas em dois conjuntos disjuntos
 - forward e back edges
-

CFG Redutíveis

- Propriedade

1. as forward edges formam um grafo acíclico no qual qualquer nó pode ser alcançado a partir do nó inicial de G
2. todas as back edges tem suas cabeças dominando suas caudas

- Verificando

- Basta remover as back-edges e checar a propriedade 1
- Computando os dominadores facilmente identificamos as back edges

CFG Redutíveis

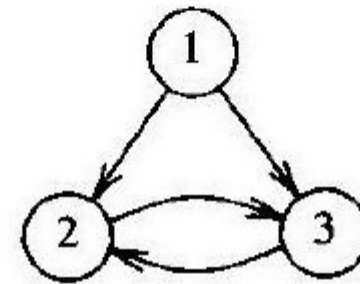
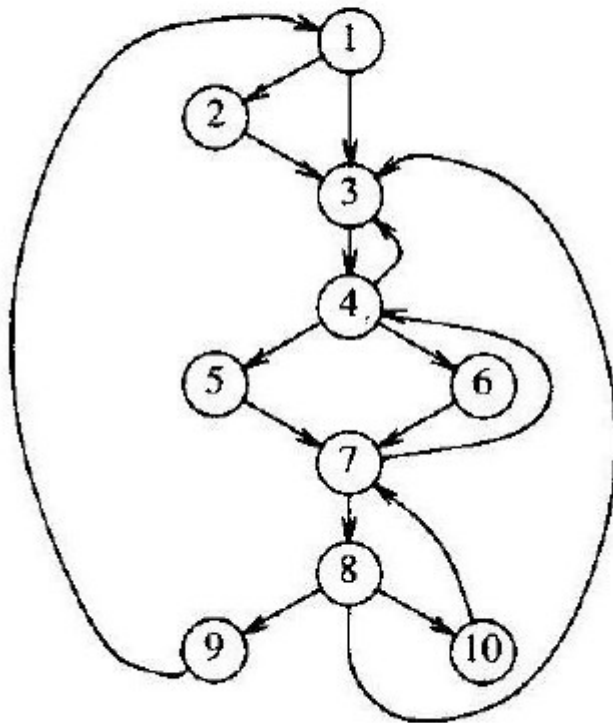


Fig. 10.18. A nonreducible flow graph.

CFG Redutíveis

- Importância
 - Precisamos apenas verificar laços naturais de back edges para termos todos os laços do programa
- Grafos como o da Fig 10.18 tornam transformações não diretamente aplicáveis
- Grafos não redutíveis são raros
 - Não aparecem se não usarmos goto

Transformações em Laços

Detecção de Invariantes do Laço

- Invariante:
 - Valor da computação não muda dentro do laço
- $s: X = y + z$
 - Dentro de um laço
 - Porém, todas as definições de y e z que alcançam s estão fora do laço
 - s é invariante
 - y e/ou z podem ser constantes
- Então s é invariante
 - $t: v = x + w$ dentro do laço
 - Invariante se w é invariante e s é a única definição de x alcançando t

Detecção de Invariantes do Laço

- Detectados usando ud-chains
- Fazendo-se várias passadas no laço, até que nenhuma sentença seja marcada como invariante
- Se temos du-chains não é necessário várias passadas

Code Motion/ Hoisting

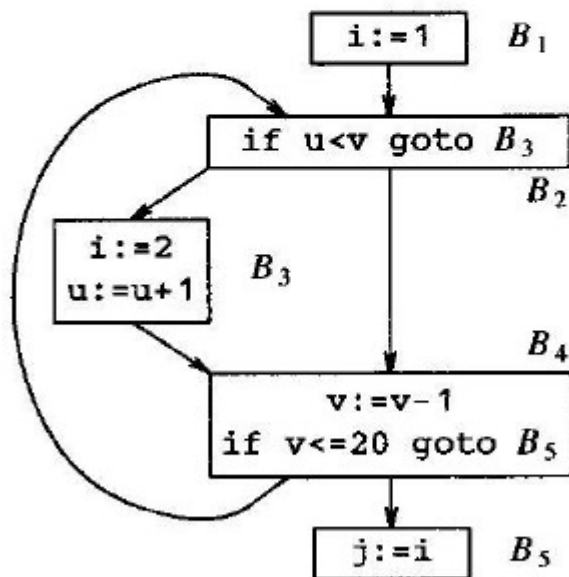
- Mover sentenças invariantes para o pre-header do laço
 - Condições
 - $s: x = y+z$ invariante
 1. O bloco de s domina todas as saídas do laço
 2. Não há outra sentença no laço que atribua a x
 3. Não há uso de x no laço alcançado por outra definição de x além de s
-

Code Motion/ Hoisting

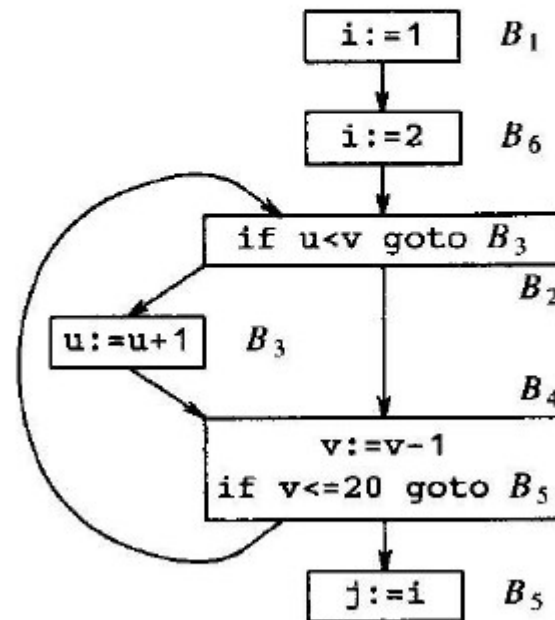
- São condições simples de verificar
- Se aplicam a programas reais
- Não são absolutamente essenciais
 - Pode-se tentar relaxá-las
- Vejamos exemplos

Code Motion

- Condição 1



(a) Before



(b) After

Code Motion

- Condição 2

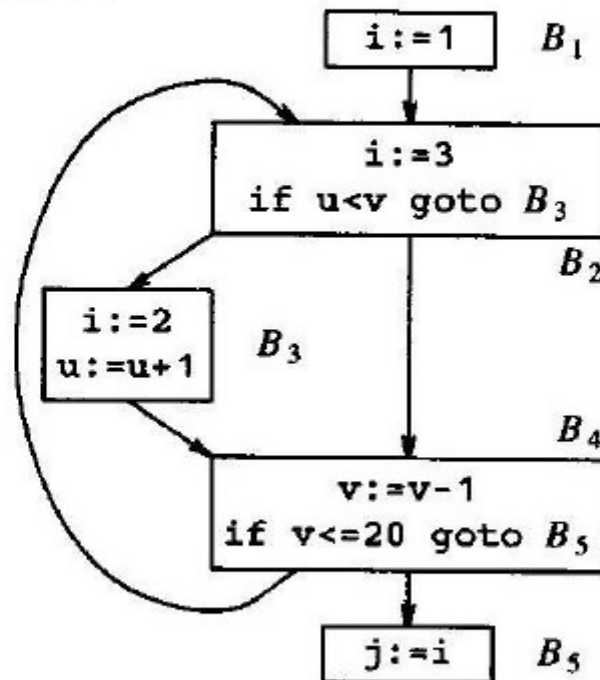


Fig. 10.37. Illustrating condition (2).

Code Motion

- Condição 3

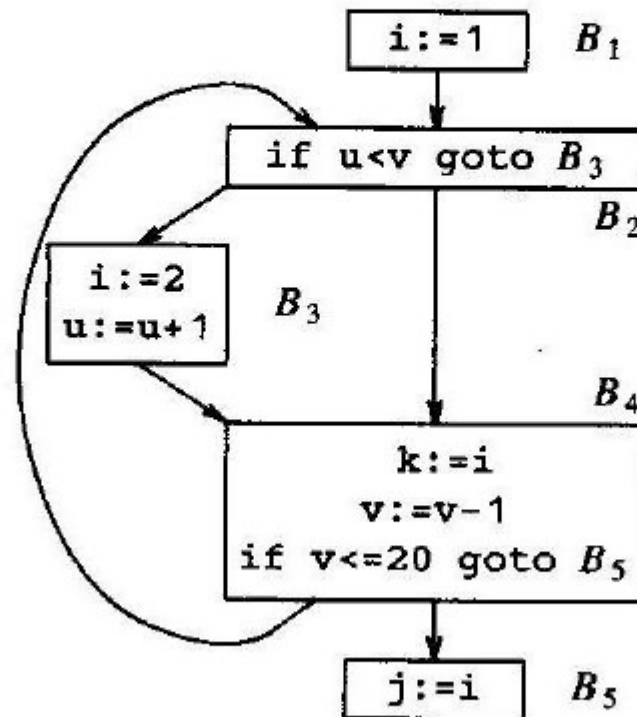


Fig. 10.38. Illustrating condition (3).

Algoritmo

1. Encontre as sentenças invariantes do laço L
 2. Para cada sentença s definindo x encontrada em (1) verifique que:
 1. Está num bloco q domina todas as saídas de L
 2. X não é definida em L
 3. Todos os usos de x em L são alcançados apenas por s
 3. Mova, na ordem dada em (1), as sentenças para o pre-header
-

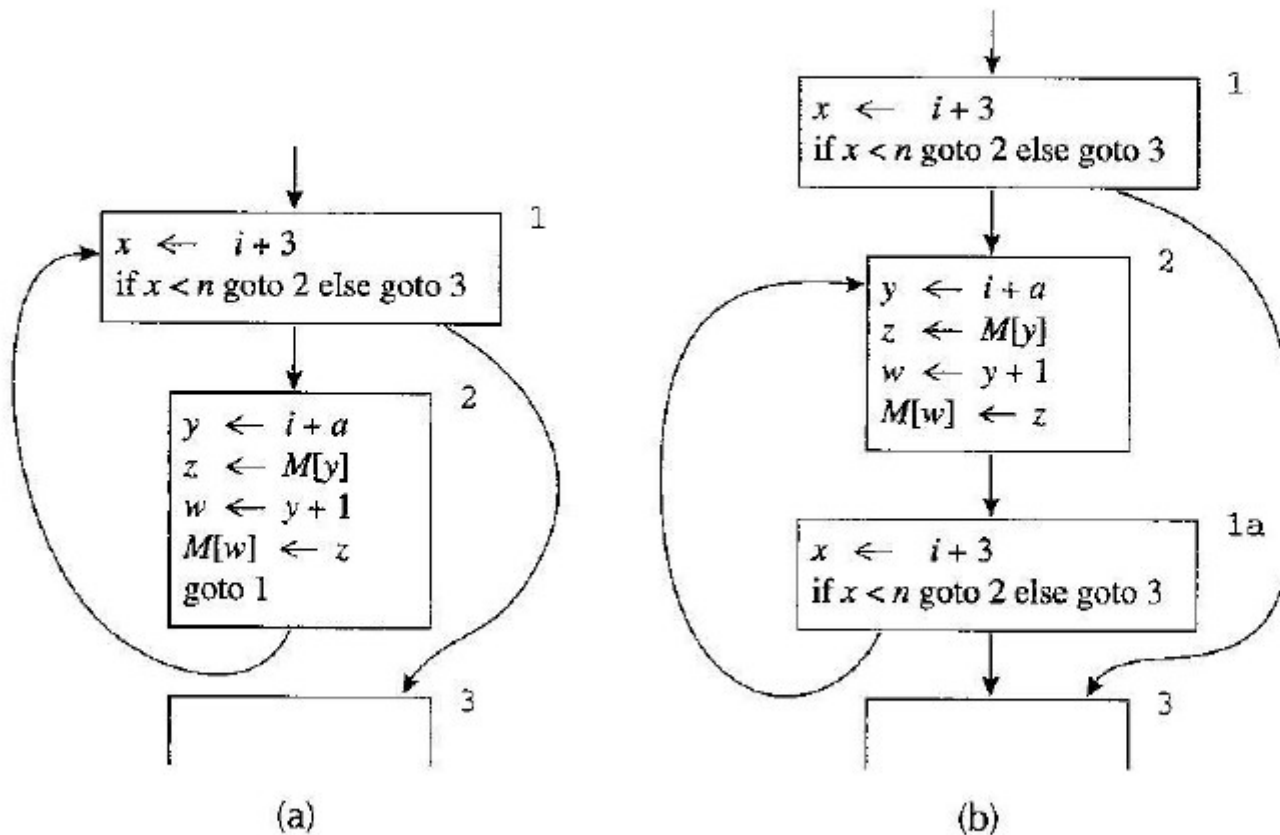
Análise das Condições

- (1) e (2) garante que o valor de x computado em s é o valor de x nas saídas de L
- (3) garante que todo uso de x em L usa o valor dado por s
- Note que garante que o tempo de execução não aumenta. Por quê?

Problema com While

- A condição (1) tende a proibir a movimentação ao pre-header de sentenças em um laço while
 - Nesse tipo de laço as sentenças tendem a não dominar todas as saídas do laço
 - Solução: transformar em repeat precedido por um if
-

Problema com While



Fonte: Appel fig 18.7

Relaxamento das Condições

- (1) pode ser relaxada
 - Não garante que o tempo de execução não aumenta
 - Fica:
 - (1') O bloco de s ou domina todas as saídas do laço ou x não é usado fora do laço
 - Em geral exige liveness analysis
 - Como o tempo de execução pode ser aumentado
 - Na média funciona bem
-

Relaxamento das Condições

- (1') pode inserir erros?
- Sim
 - Pense em x/y com um teste sobre $y=0$
 - Movendo x/y para o pre-header pode causar erro
- Para usar (1')
 - Otimização deve poder ser desligada pelo programador
 - Não aplicar para divisões

Relaxamento das Condições

- Mesmo se nenhuma das 3 condições do passo 2 forem satisfeitas poderíamos mover $x = y+z$ fora do laço
- Coloque $t=y+z$ no pre-header
- Troque $x = y+z$ por $x = t$ no laço
- Deixe para copy propagation

Variáveis de Indução

- Variável de indução em um laço L
 - Toda vez que muda de valor é
 - Decremento ou incremento por constante
 - Pode ser mais de uma vez
 - Pode ser # diferente de mudanças a cada iteração
 - Variáveis de indução básicas
 - Somente atribuições da forma $i = i \pm c$
 - C é constante
-

Variáveis de Indução

- Variáveis adicionais (ou dependentes)
 - Variável j definida de forma única e dentro de L
 - Valor é função linear de uma variável básica i , onde j é definida
 - $j = c \cdot i + d$ (c e d constantes)
 - Tripla (i, c, d)
 - Diz-se que j pertence à família de i
 - Por definição, i pertence à sua própria família
 - Tripla $(i, 1, 0)$

Detecção de Variáveis de Indução

- Informações necessárias
 - Laço L
 - Reaching Definitions
 - Invariantes do laço computadas
- Método
 1. Encontre as variáveis básicas de indução, associando triplas do tipo $(i, 1, 0)$

Detecção de Variáveis de Indução

- Método

2. Encontre k tal que k tenha uma definição em L de uma das formas:

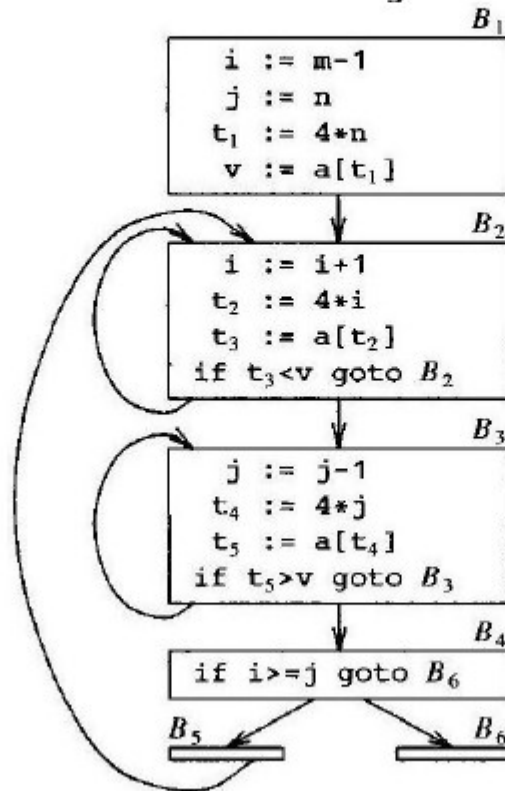
- $K = j*b$, $k=b*j$, $k=j/b$, $k= j\pm b$, $k= b\pm j$ (b constante e j variável de indução, básica ou não)
- Se j é básica, k é da família de j
 - Se $k = j*b$ a tripla é $(j,b,0)$
- Senão, seja j da família de i . Devemos ter:
 - Não há atribuição a i entre a atribuição a j em L e a atribuição a k ;
 - Nenhuma definição de j de fora do laço alcança k

Detecção de Variáveis de Indução

- No caso comum
 - j e k são temporários no mesmo bloco, fazendo as condições anteriores fáceis de checar
- No caso geral
 - Reaching definitions fornece o necessário se analisarmos o CFG do laço L
- Tripla: se tripla de $j = (i, c, d)$ e $k = b * j$
 - $k: (i, b * c, b * d)$ com b, c, d constantes

Exemplo

- Quais as variáveis de indução?



(a) Before

Strength Reduction

- Informações

- Laço L
- Reaching Definitions
- Famílias de variáveis de indução

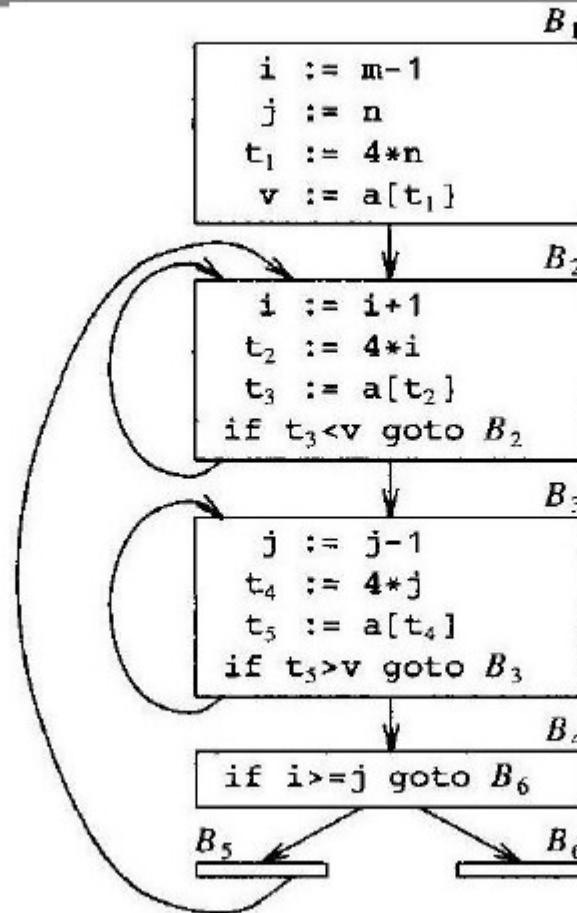
- Método

- Considere cada variável de indução básica i
 - Para cada j na família de i com tripla (i,c,d)
 - Crie uma nova variável s
 - Se j_1 e j_2 têm a mesma tripla crie apenas uma para ambos
 - Troque a atribuição a j por $j=s$
-

Strength Reduction

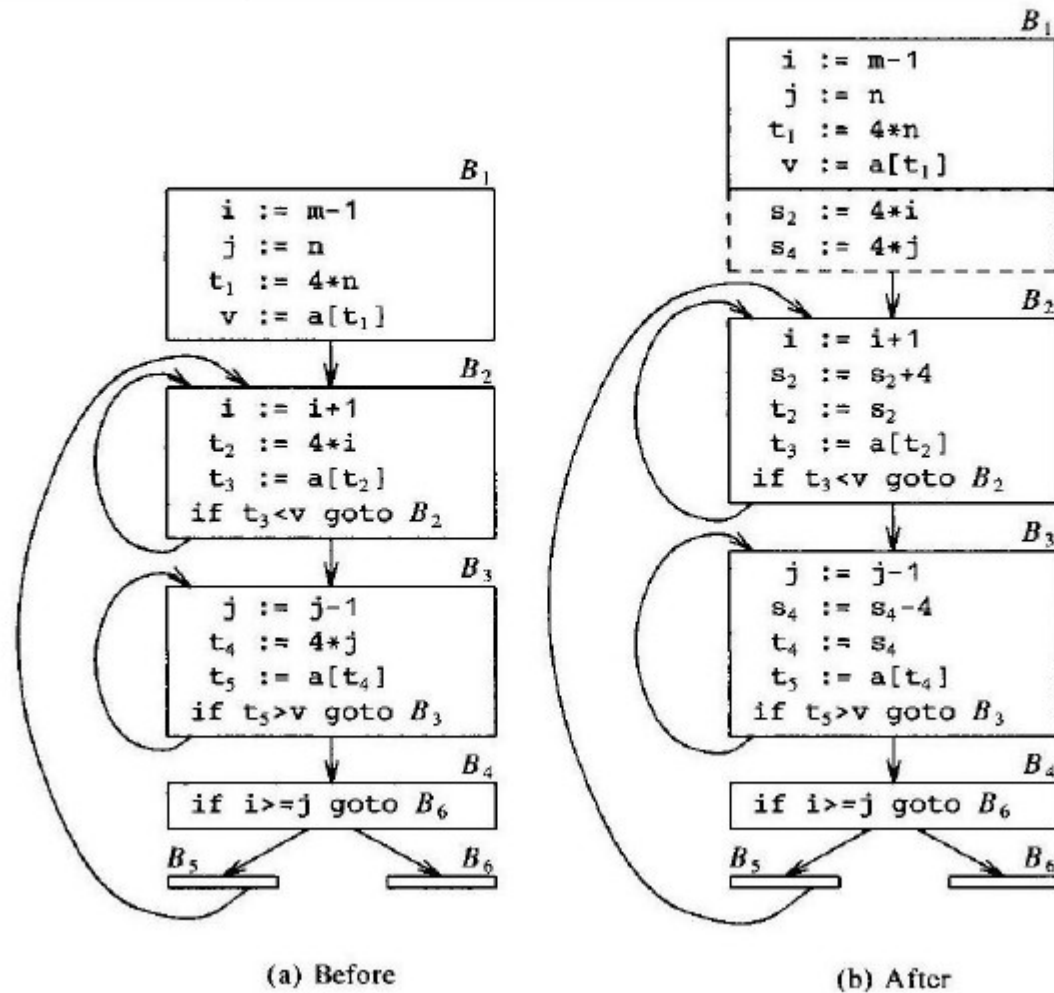
- Método (continuação)
 - Imediatamente após $i = i + n$, n constante, adicione
 - » $s = s + c * n$ ($c * n$ é constante)
 - Coloque s na família de i com tripla (i, c, d)
 - Falta garantir que $s = c * i + d$ no início do laço
 - No final do pre-header coloque
 - $s = c * i$ ($s = i$ se $c = 1$)
 - $s = s + d$ (omitir se $d = 0$)

Exemplo



(a) Before

Strength Reduction Aplicada



Eliminação de Variáveis de Indução

- Variáveis de indução básica (i)
 - Únicos usos são para computar outras variáveis de indução
 - Ou condições em desvios
- Seja j uma variável da família de i
 - Podemos substituir i por j (i,c,d) nos testes
 - Assim acabamos eliminando i
 - É melhor escolher j com c e d simples
 - Ex: (i,1,0)

Eliminação de Variáveis de Indução

- Cuidado com constantes negativas
 - $i \geq j$ é equivalente a $t \geq 4*j$
 - $i \geq j$ é equivalente a $t \leq -4*j$
- Assumindo c positivo
 - If i relop x goto B , x não é v.i. Substitui por:
 - $r = c*x$
 - $r = r + d$
 - If j relop r goto B
 - r é um novo temporário
 - O teste x relop i é tratado de forma análoga

Eliminação de Variáveis de Indução

- Se temos 2 v. i.:
 - If i1 relop i2 goto B.
 - Verificamos se ambas i1 e i2 podem ser removidas
 - O caso bom é quando temos
 - $j1 = (i1, c1, d1)$ e $j2 = (i2, c2, d2)$
 - E $c2 = c1$ e $d1 = d2$
 - Assim $i1 \text{ relop } i2$ é equivalente a $j1 \text{ relop } j2$
 - Outros casos podem não valer à pena
 - Introduz duas multiplicações e uma adição
 - Elimina apenas 2 passos

Eliminação de Variáveis de Indução

- Finalmente:
 - Elimine todas as atribuições às v.i. básicas em L
 - Tornaram-se inúteis
- Considere sentenças do tipo
 - $j = s$, introduzidas pela strength reduction
 - Verifique que não existem atribuições a s entre $j=s$ e o uso de j
 - Pode necessitar de reaching definitions
 - Troque os usos de j por usos de s e apague $j=s$

Exemplos

- Como fica a eliminação de v. i. nos laços ao redor de
 - B2 e B3
 - Quais as variáveis de indução?
 - Podem ser eliminadas?
 - B2, B3, B4 e B5
 - Examine i , j , s_2 e s_4